



PROGRAMMING FUNDAMENTALS.

C++ I.

Create a lab report for the following programming challenge:

C++ Final Project Instructions

 **Due Date: April 26, 2026 – No Late Submissions Accepted**

 **Submit Exactly Two Files:**

- `final.cpp` — Your complete and working C++ source file (all code in one file)
- `final.pdf` — Your professional lab report using the Blackboard template

✗ ZIP files or compressed submissions will not be accepted or graded.

Final Project: Homeowner Expenses Calculator

You are required to design and implement a **modular, file-driven expense management system** in C++. The program will help homeowners track, analyze, and report on monthly expenses throughout the year.

Project Summary

You are building a **console application** that:

- Loads expense data from a file
- Organizes it into structured memory
- Analyzes totals, savings, and over-budget items
- Sorts and reports data
- Outputs meaningful summaries based on user selections

-  **Do not prompt the user to enter expenses manually.**
 -  Your program must load all expenses from a file using file input operations.
-

Functional Requirements

Your program must do **all** of the following:

1. Read Expenses from a File

- Read a file (`expenses.txt`) with the following format:
- `<month> <category> <amount> <budgeted>`
- Example line:
`3 Utilities 350 300`

2. Store Data Using a Struct

Define a structure to hold each expense:

```
struct Expense {  
    int month;           // 1-12  
    std::string category; // no spaces  
    double amount;       // actual spending  
    double budgeted;     // budgeted amount  
};
```

Store the data in an array or vector of these structs.

3. Display All Expenses Grouped by Month

- Show expenses for each month
- Include both actual and budgeted amounts

4. Find and Display the Month with the Highest Total Expenses

- Compute monthly totals
- Identify the month with the highest spending.

5. Sort and Display All Monthly Totals

- Sort from highest to lowest and lowest to highest

6. Show Detailed Report for a User-Specified Month

- Prompt user for a **month number (1–12)**
- Show:
 - All categories
 - Spent vs. budgeted
 - Over-budget items
 - Savings

7. Input Validation

- Reject:
 - Invalid months (not in 1–12)
 - Negative values in file
- Validate input when asking for month number for report

8. Programming Expectations

- Use:
 - **Functions** to divide tasks logically
 - **Structs** for data representation
 - **Pointers** where efficient (e.g., sorting, totals)
 - Your program must be contained entirely in one file: `final.cpp`
-

Required File Format — `expenses.txt`

All data for this program must come from this file. Do **not** ask the user to enter values at runtime.

Sample `expenses.txt` Content:

```
1 Mortgage 1200 1200
1 Utilities 350 300
2 Groceries 500 450
2 Gas 120 100
3 Internet 80 80
3 Insurance 220 200
4 Dining 250 200
5 Travel 1000 800
6 Health 200 180
```

Sample Output – Detailed Month Report

```
=== Expense Report for Month: 2 (February) ===  
Category: Groceries | Spent: $500.00 | Budgeted: $450.00 | Over-Budget by  
$50.00  
Category: Gas      | Spent: $120.00 | Budgeted: $100.00 | Over-Budget by  
$20.00  
Total Over-Budget: $70.00  
Savings: $0.00
```

Sample Output – Monthly Totals Sorted

```
=== Monthly Totals (Descending) ===  
May: $1000.00  
January: $1550.00  
March: $300.00  
...
```

Lab Report Guidelines (`final.pdf`)

Your report must:

- Use the **Blackboard-provided template**
- Be formatted professionally
- Include proof of every requirement using:
 -  A short explanation
 -  A **screenshot** showing relevant output

Report Sections:

1. **Title Page**
2. **Overview of the Program**
3. **List of Project Requirements**
4. **Screenshots + Commentary**
 - One for each feature (e.g., file reading, sorting, over-budget detection)
 - Do **not** include long, cluttered screenshots
5. **Design Discussion**
 - Brief notes on structure, challenges, and functions
6. **Conclusion**
 - What you learned, what could be improved

 Lack of readability or missing sections will result in a 10-point deduction.

Submission Instructions

Submit these two files to Blackboard:

File	Description
<code>final.cpp</code>	Your complete C++ program source code
<code>final.pdf</code>	Your formatted and complete lab report

✗ DO NOT submit:

- Any .zip, .rar, or compressed file
- Multiple source files
- Separate images or folders

✗ Submissions not following naming rules or format will **not be accepted**.

Grading Rubric (100 Points)

Category	Points
All Functional Requirements Met	30 pts
Structs, Functions, Modularity	15 pts
Input Validation and Reporting	10 pts
File Reading and Analysis Logic	10 pts
Use of Pointers/Advanced Techniques	10 pts
Lab Report – Screenshots & Commentary	15 pts
Lab Report – Presentation & Formatting	10 pts

Final Checklist

- ☒ `final.cpp` includes all required logic
- ☒ `final.pdf` shows proof of each requirement
- ☒ All data is **read from file only**
- ☒ Code is organized and commented

- ☒ No cluttered screenshots
 - ☒ Submitted on time via Blackboard
 - ☒ **No ZIP files or multiple uploads**
-

You must submit two files.

- A pdf document, the lab report. Name it final.pdf.
- Final.cpp with the modular code of your program.

For your reference, there is a lab report template available in the Blackboard system under Course Content.

Any question? Just email me.

LDV.